

be-engine — Architecture Analysis

A review of the current state, concrete problems, and recommended direction.

What's genuinely good (don't touch)

- **The RHI seam is clean.** `sen-rhi` is handle-based (`SenTexture{id}` etc.), the backend is a swappable facade, and core never sees a `Vk*` type. This is the single most valuable thing in the codebase and it's done right.
- **Modern Vulkan baseline.** You're on 1.3 core with `dynamicRendering + synchronization2` (`SenVulkanBackend_Core.cpp:110`). No `renderpass/framebuffer` objects, `VkCmdPipelineBarrier2` everywhere. This is the correct 2026 baseline and it saves you a mountain of legacy plumbing.
- **The shader pipeline is the real differentiator.** Runtime Slang compilation, hot reload, the `@be-shader/@be-material` DSL, and the autogen boilerplate CLI together make this a genuinely pleasant place to prototype. That's your moat — most hobby engines have none of it.
- **SRM-as-example, not engine.** Keeping the deferred pipeline in `toolkit/` as a composable thing rather than baking it into core is a mature decision.

The one structural ceiling (you already know this)

`BeRenderer::Render()` (`BeRenderer.cpp:66`) is a dumb front-to-back walk of `vector<BeRenderPass*>`. Dependencies exist only as **call order in SRM**, and barriers are **re-active**: `TransitionTextures` lazily flips per-mip layouts (`SenVulkanCommandBuffer.cpp:121`) and it only works because execution is strictly linear and single-queue. This is correct today and a dead end tomorrow — it cannot express aliasing, multi-queue, or cull dead passes. Your instinct to evolve this into a `BeRenderGraph` with `Setup()`-declared reads/writes is the right north star. I'd validate that whole plan as-is.

Concrete bugs/gaps I'd fix *before* the graph refactor

These are cheap, and you want a non-crashing, debuggable base under a long refactor:

1. **No swapchain-resize / out-of-date handling.** `VkAcquireNextImageKHR` and `VkQueuePresentKHR` return values are ignored (`SenVulkanBackend_Swapchain.cpp:222,259`). `ResizeSwapchain` exists but nothing calls it on `VK_ERROR_OUT_OF_DATE_KHR`. A window resize → validation spam / stale extent / eventual crash. This is a "first thing a new user hits" bug.
2. **Debug markers are no-ops.** `Render()` wraps every pass in `BeginDebugEvent/EndDebugEvent` (`BeRenderer.cpp:74`)... and the bodies are empty (`SenVulkanBackend_Core.cpp:272-276`). So `RenderDoc/Nsight` show unlabeled soup and no object names. For a *graphics programmer's workbench*, this is the highest value-to-effort fix in the repo: wire `VkCmdBeginDebugUtilsLabelEXT + VkDebugUtilsObjectNameInfoEXT`.

3. **Handles have no generation bits.** Registries are monotonic-ID maps (SenVulkanBackend_Core.cpp:23-29). Stale handles `.at()`-throw rather than being caught; no use-after-free detection. A `{index, generation}` handle would make the RHI self-validating — worth it before the graph starts juggling transient resources.
4. **Blind physical-device pick.** `_physicalDevice = devices[0]` (Core.cpp:72) with no discrete-GPU preference or feature-support check. Fine on your machine, lands on the iGPU for someone else.

Performance levers (in priority order)

5. **Single frame in flight** is your biggest cost. One `_frameCmd`, one `InFlightFence`, and `BeginFrame` waits on it at the top of *every* frame (Swapchain.cpp:216). CPU and GPU are fully serialized — zero overlap. Introducing N frames-in-flight (per-frame command buffer + pool + sync + per-frame dynamic uniform buffers) is the single highest-impact change you can make and it's a prerequisite for the graph anyway.
6. **Single queue.** One graphics queue; no transfer or compute family enumerated (Core.cpp:79). Every upload stalls graphics, and it blocks compute passes entirely.
7. **Every device upload is a create-fence → submit → wait → destroy stall** (SenVulkanBackend_Buffer.cpp:89, `SubmitImmediate` in Core.cpp:231). Fine per-resource at load; but loading a scene = a stall per mesh/texture with no batching. A transfer queue + persistent staging ring fixes this.
8. **Zero threading** (no `std::thread/mutex/async` anywhere in core/toolkit). Slang compilation, asset loading, and command recording are all on the main thread. Multithreaded pass recording is a natural payoff of the graph.

Where I'd point the engine

Your own 5-step plan (queues+FIF → real `SenCommandBuffer` objects → transfer ring → `BeRenderGraph` → compute) is sound. I'd only adjust the **sequencing** and resolve the open `SenDevice` question:

- **Do #1–2 above first (resize + debug markers).** A day of work; makes everything after it debuggable and demo-safe.
- **Then frames-in-flight (#5).** Biggest perf win, and it forces the “real `SenCommandBuffer` objects” step anyway.
- **Introduce `SenDevice` — yes, do it.** The static `SenVulkanBackend::Foo()` facade will fight you the moment the graph needs explicit context (transient pools, per-frame allocators, per-queue submission). A `SenDevice` object also unlocks testing and a second backend. Bite it before the graph, not after.
- **Then the render graph, then compute.** Compute genuinely does fall out for free once queues + graph exist — and *compute is half of every modern technique* (SSAO, GTAO, clustered light culling, your GGX prefilter/BRDF LUT IBL work), so it's what actually pays for the whole refactor.

Strategic framing: don't let entt/scenes tempt this toward being a "game engine" — that's not where the value is. Lean into what already makes it special: the runtime-shader DX. The engine wins by being *the fastest place to prototype a rendering technique*. Concretely that means (a) the render graph so techniques compose without manual barrier surgery, (b) compute, (c) in-engine GPU timestamp profiling, and (d) a live shader/material inspector on the imgui layer. Those four turn "a nice Vulkan renderer" into "a workbench I reach for instead of Shadertoy + a scratch project."